Batch 1 Implementation Plan — AI Challenge HCMC 2026

Overview

Build a **working end-to-end retrieval system** for the 3 confirmed qualification tasks (**KIS-T**, **Q&A/VQA**, **TRAKE**) using the Batch 1 dataset (L21–L30, 873 videos across 10 groups). The architecture follows the **TycheVid 2024 pipeline** as its core, adapted for sousveillance data, and is structured to scale automatically as more local data is downloaded.

> **Strategy**: Batch 1 ships with pre-extracted assets (Keyframes, CLIP features, Objects, Map-keyframes). We **use these directly** as the baseline — no re-extraction needed. This gets us a working system fast. The stronger model upgrades (SigLIP2, InternVL3.5, etc.) come after the core pipeline is proven.

Competition Deadlines

Phase	Deadline	Target
Phase 1	21 August 2026	**Full Working Pipeline** (KIS-T, VQA, TRAKE) + UI + Temporal search
Phase 2	28 August 2026	Bug fixes, score fusion tuning, VLM prompt refinements
Phase 3	04 September 2026	Final polish, submission generation

---

GPU Constraints

Resource	VRAM	Status	Best used for
RTX 4060 Laptop	8 GB	Local, always available	CLIP encoding, FAISS indexing, 4-bit VLM inference
Colab T4	15 GB	Free-tier, limited sessions	8-bit VLM inference, heavy batch jobs

Resource	VRAM	Status	Best used for
Rented GPU	TBD	On-demand if needed	Full-scale stress tests, better VLMs

Implication: VLMs must be **4-bit quantized** to run locally. Use `bitsandbytes` for quantization. For better quality (8-bit), offload to Colab T4.

Data Layout (Confirmed)

All assets follow a consistent naming convention: **one file/folder per video**, named `LXX_Vyyy`.

Asset	Location	Format	Availability
Videos	`data/raw_videos/video_L21/L21_Vxxx.mp4`	MP4	✅ L21 only (29 videos); L22–L30 to be downloaded
Keyframes	`data/keyframes/LXX_Vxxx/NNN.jpg`	JPEG, 1-indexed, 3-digit	✅ L21 only (locally); objects dir has L21–L30 partially
CLIP features	`data/clip-features-32/LXX_Vxxx.npy`	**One `.npy` per video**, shape `[N, 512]` float32	✅ All 873 files present (L21–L30)
Map-keyframes	`data/map-keyframes/LXX_Vxxx.csv`	**CSV per video**, headers: `n, pts_time, fps, frame_idx`	✅ All 873 files present (L21–L30)
Objects	`data/objects/LXX_Vxxx/NNN.json`	JSON per keyframe (see format below)	✅ 630 folders present (partial)
Media info	`data/media-info/`	JSON per video (YouTube metadata)	⏳ To be confirmed

Map-keyframes CSV format (confirmed from `L21_V001.csv`)

```
n,pts_time,fps,frame_idx
1,0.0,30.0,0
2,3.0,30.0,90
3,8.7,30.0,261
```

- `n` — keyframe sequence number, **1-indexed**. Matches filename: n=1 → `001.jpg`, n=307 → `307.jpg`
- `pts_time` — timestamp in seconds within the video
- `fps` — video frame rate
- `frame_idx` — **original frame index in the raw MP4 — this is the value submitted to the competition**

Lookup formula: Given keyframe file `"001.jpg"` → `n = int("001") = 1` → CSV row where `n == 1` → `frame_id = frame_idx` from that row → submit `frame_id`.

Object JSON format (confirmed from `L21_V001/001.json`)

```
{
  "detection_scores": ["0.79673874", "0.6866252", ...],
  "detection_class_names": ["/m/01jfsr", "/m/079cl", ...],
  "detection_class_entities": ["Lantern", "Skyscraper", ...],
  "detection_boxes": [["0.468", "0.366", "0.636", "0.467"], ...],
  "detection_class_labels": ["84", "379", ...]
}
```

Critical: All numeric values are **strings** — must cast with `float()` when loading. Use `detection_class_entities` for human-readable labels. `detection_boxes` format: `[ymin, xmin, ymax, xmax]` normalized 0–1.

CLIP features `.npy` (confirmed from `clip-features-32/`)

- **Folder name** `clip-features-32` confirms the model is CLIP ViT-B/**32**.
- One `.npy` file per video: `L21_V001.npy`, shape `[N, 512]` float32, where N = number of keyframes.
- `npy_row_idx = n - 1` (CSV `n` is 1-indexed; numpy is 0-indexed).
- Lookup: `features[npy_row_idx]` = CLIP vector for keyframe `n`.

Target Directory Layout (after ingestion)

```
AI_Challenge_26/
├── data/
│   ├── raw_videos/video_L21/         # L21_V001.mp4 … L21_V031.mp4
│   ├── keyframes/LXX_Vxxx/           # 001.jpg … NNN.jpg
│   ├── clip-features-32/             # LXX_Vxxx.npy  (one per video, shape [N, 512])
│   ├── map-keyframes/                # LXX_Vxxx.csv  (n, pts_time, fps, frame_idx)
│   ├── objects/LXX_Vxxx/             # NNN.json  (object detections per keyframe)
│   └── media-info/                   # LXX_Vxxx.json  (YouTube metadata, TBC)
├── indexes/                          # Built by ingestion (add to .gitignore)
│   ├── faiss_clip.index              # FAISS IndexFlatIP with all CLIP vectors
│   ├── metadata.parquet              # Master keyframe registry (central table)
│   ├── objects.parquet               # Flat object detections table
│   └── media_info.json               # Dict: video_id -> metadata (optional)
├── src/
│   ├── config.py
│   ├── offline_pipeline/
│   │   ├── metadata_loader.py        # Build metadata.parquet from CSVs
│   │   ├── build_index.py            # Build faiss_clip.index from .npy files
│   │   ├── objects_loader.py         # Build objects.parquet from JSONs
│   │   └── media_loader.py           # Build media_info.json (optional)
│   ├── online_pipeline/
│   │   ├── query_encoder.py          # Encode text query to CLIP vector
│   │   ├── retrieval.py              # FAISS search + metadata join
│   │   ├── object_filter.py          # Re-score by detected objects
│   │   ├── fusion.py                 # Weighted score combination
│   │   └── temporal_search.py        # TycheVid temporal re-ranking
│   ├── tasks/
│   │   ├── kis_t.py
│   │   ├── vqa.py
│   │   └── trake.py
│   └── utils/
│       ├── video_utils.py
│       └── metrics.py
├── app.py                            # Streamlit UI entry point
└── main_ingest.py                    # Offline ingestion entry point
```

Environment Setup

Conda environment: `aic26` | **Python**: 3.11 | **Fresh install** (no packages yet)

Step 0-A: Activate

```
conda activate aic26
```

Step 0-B: Core install (CPU — runs without GPU for indexing/retrieval)

```
# Core scientific stack
pip install numpy>=1.26.0 scipy>=1.11.0

# Image/video processing
pip install opencv-python>=4.8.0 Pillow>=10.0.0

# Data processing
pip install polars>=0.20.0 tqdm>=4.66.0

# Vector search — FAISS CPU (sufficient at this scale)
pip install faiss-cpu>=1.8.0

# CLIP embedding model — ViT-B/32 to match provided features
pip install open_clip_torch>=2.24.0

# PyTorch — adjust cu121 to match your installed CUDA version
pip install torch>=2.1.0 torchvision>=0.16.0 --index-url https://download.pytorch.org/whl/cu12

# UI
pip install streamlit>=1.31.0

# API server (for future backend)
pip install fastapi>=0.109.0 uvicorn>=0.27.0

# Utilities
pip install requests python-dotenv
```

Step 0-C: GPU additions (for VQA — install when ready)

```
# VLM with 4-bit quantization support (fits in 8GB VRAM)
pip install transformers>=4.40.0 accelerate>=0.26.0 sentencepiece bitsandbytes>=0.43.0

# Optional: faster inference
pip install vllm>=0.4.0
```

Phase 1: Data Ingestion & Indexing (Offline — Run Once)

1.1 Parse Map-Keyframes → Master Metadata Table

File: `src/offline_pipeline/metadata_loader.py`

Build a master Polars DataFrame — the **central registry** for everything. Every row = one keyframe.

Column	Type	Source	Description
`video_id`	str	CSV filename stem	e.g., `"L21_V001"`
`keyframe_name`	str	`str(n).zfill(3)`	e.g., `"001"`
`pts_time`	float	CSV `pts_time`	Timestamp in seconds
`frame_id`	int	CSV `frame_idx`	**Original MP4 frame index — the value submitted to competition**
`keyframe_path`	str	constructed	Absolute path to JPEG (may not exist if not downloaded)
`npy_row_idx`	int	`n - 1`	Row index in this video's `.npy` file
`faiss_idx`	int	global counter	Row index in the concatenated FAISS index
`video_path`	str	constructed	Absolute path to MP4 (may not exist if not downloaded)

Logic: 1. Iterate all CSVs in `data/map-keyframes/` in **sorted alphabetical order** (guarantees consistent FAISS index). 2. For each CSV: `df = polars.read_csv(path)` — columns are `n, pts_time, fps, frame_idx` . 3. Add computed columns: `keyframe_name = str(n).zfill(3)` , `npy_row_idx = n - 1` . 4. Assign `faiss_idx` as a running global counter across all videos. 5. Concatenate all DataFrames and save to `indexes/metadata.parquet` .

Verification: After building, check that `faiss_idx` values are 0, 1, 2, ... N-1 with no gaps. The maximum `faiss_idx` + 1 must equal the total number of CLIP vectors across all `.npy` files.

1.2 Build FAISS Index from CLIP Features

File: `src/offline_pipeline/build_index.py`

```
Per-video .npy files (sorted) → concatenate → L2-normalize → FAISS IndexFlatIP → save
```

Steps: 1. Iterate `data/clip-features-32/LXX_Vxxx.npy` files in **same sorted order as Step 1.1**. 2. For each: `arr = np.load(path).astype('float32')` — shape `[N_i, 512]` . 3. Concatenate: `all_features = np.concatenate([arr1, arr2, ...], axis=0)` . 4. L2-normalize: `all_features /= np.linalg.norm(all_features, axis=1, keepdims=True)` . 5. Build: `index = faiss.IndexFlatIP(512); index.add(all_features)` . 6. Save: `faiss.write_index(index, str(FAISS_INDEX_PATH))` .

Scale: 873 videos × ~150 keyframes avg ≈ 130,000 vectors. FAISS flat IP memory: ~130K × 512 × 4 bytes ≈ 267 MB — fine for laptop RAM.

Lookup at search time: `faiss_idx` from metadata table → `all_features[faiss_idx]` → FAISS result.

1.3 Load Object Detections → Polars Store

File: `src/offline_pipeline/objects_loader.py`

Parse all object JSON files into a flat Polars DataFrame. One row per detected object instance.

Column	Type	Description
`video_id`	str	e.g., `"L21_V001"`
`keyframe_name`	str	e.g., `"001"`
`faiss_idx`	int	joined from metadata for fast candidate lookup
`object_label`	str	from `detection_class_entities`
`confidence`	float	from `detection_scores` (cast from string)
`bbox_ymin`	float	from `detection_boxes[i][0]` (cast from string)
`bbox_xmin`	float	from `detection_boxes[i][1]`
`bbox_ymax`	float	from `detection_boxes[i][2]`
`bbox_xmax`	float	from `detection_boxes[i][3]`

Parsing note: All values in the JSON are strings — cast with `float()`:

```
scores = [float(s) for s in data["detection_scores"]]
labels = data["detection_class_entities"]
boxes  = [[float(v) for v in box] for box in data["detection_boxes"]]
```

Filter: discard detections with `confidence < 0.3`. Skip video folders missing from `data/objects/` gracefully (630 of 873 are present). Save to `indexes/objects.parquet`.

1.4 Entry Point: `main_ingest.py`

Orchestrates phases 1.1 → 1.3 in order. Prints a summary:

```
=== Ingestion Complete ===
Videos processed:   873
```

```
Total keyframes:    ~130,000
FAISS index size:   ~130,000 vectors (512-dim)
Objects rows:       ~5,000,000 (varies)
```

Run:

```
conda run -n aic26 python main_ingest.py
```

Verification gate: Must complete without errors before starting online pipeline.

Phase 2: Online Query Pipeline

2.1 Query Encoder

File: `src/online_pipeline/query_encoder.py`

The folder name `clip-features-32` confirms the model: CLIP ViT-B/**32**.

```
import open_clip, torch, numpy as np

model, _, _ = open_clip.create_model_and_transforms('ViT-B-32', pretrained='openai')
tokenizer   = open_clip.get_tokenizer('ViT-B-32')
model.eval()

def encode_text(query: str) -> np.ndarray:
    tokens = tokenizer([query])
    with torch.no_grad():
        features = model.encode_text(tokens).float().numpy()
    features /= np.linalg.norm(features, axis=1, keepdims=True)
    return features[0]  # shape: (512,)
```

> **CRITICAL**: `pretrained='openai'` must be used. Any other pretrained weight produces an incompatible embedding space — searches will return garbage.

GPU: CLIP ViT-B/32 is ~350 MB. Fits on RTX 4060 8GB. Add `.cuda()` for faster encoding.

2.2 FAISS Search

File: `src/online_pipeline/retrieval.py`

Load FAISS index and metadata parquet once at startup (kept in memory).

```
def search_clip(query_vector: np.ndarray, top_k: int = 100) -> list[dict]:
    """
    Returns top_k results:
    [{"video_id": ..., "keyframe_name": ..., "frame_id": ...,
      "pts_time": ..., "score": ..., "keyframe_path": ..., "faiss_idx": ...}, ...]
    Joins FAISS result indices with metadata parquet via faiss_idx column.
    """
```

For TRAKE Phase B (search within a specific video):

```
def search_clip_in_video(query_vector: np.ndarray, video_id: str, top_k: int = 5) -> list[dict
    """Filter metadata to target video, then search only those faiss_idx rows."""
```

2.3 Object Filter

File: `src/online_pipeline/object_filter.py`

Re-scores candidates based on object detections. Applied **after** FAISS search, not as a pre-filter.

```
def filter_by_objects(
    candidates: list[dict],
    required_objects: list[str],    # e.g., ["Person", "Car"]
    mode: str = "boost"
) -> list[dict]:
    """
    mode='boost': add bonus score to candidates that have required objects
    mode='hard':  remove candidates missing any required object
    """
```

2.4 Score Fusion

File: `src/online_pipeline/fusion.py`

For Batch 1 (single embedding model + object filter):

```
final_score = alpha * clip_cosine_score + beta * object_boost
```

Design to accept a `list[(model_name, scores)]` for easy RRF upgrade when adding a second model:

```
# RRF formula (for future multi-model use):
# rrf_score(d) = sum_model [ 1 / (k + rank_model(d)) ]  where k=60
```

Default weights: `alpha=0.7, beta=0.3`

2.5 Advanced Temporal Search

File: `src/online_pipeline/temporal_search.py`

Adapted from TycheVid's core contribution. Activates only when a query contains temporal keywords.

Algorithm: 1. Scan query for temporal keywords (see `TEMPORAL_KEYWORDS` in config). 2. If found AND query has ≥ 2 sub-events: split query on temporal keywords. 3. For each sub-event: `search_clip()` independently → top-K candidates. 4. For each candidate pair: check temporal ordering within `±TEMPORAL_WINDOW` keyframe positions. 5. Boost pairs in correct order; penalize reversed pairs. 6. Return re-ranked combined results.

Window size: `±20 keyframes` (sousveillance is slower-paced than TycheVid's news footage). **Trigger condition**: temporal keyword detected **AND** ≥ 2 extractable sub-events. Otherwise: skip and return fusion results directly.

Phase 3: Task-Specific Heads

New directory: `src/tasks/`

Task 1: KIS-T (`src/tasks/kis_t.py`)

Input: Natural-language description of a video segment. **Output format**: `video_id, frame_id`

```
query text
  → encode_text(query)
  → search_clip(top_k=100)
  → filter_by_objects(mode="boost")   [if objects detected in query text]
  → temporal_search()                  [if temporal keywords detected]
  → sort by final_score descending
  → return top-100 as [(video_id, frame_id, score), ...]
```

> **CRITICAL**: `frame_id` to submit = CSV `frame_idx` column — the **original MP4 frame index**. It is NOT the 3-digit keyframe filename integer.
>
> Example from `L21_V001.csv` : keyframe `"002"` (n=2) maps to `frame_idx=90` . Submit: `L21_V001, 90` — not `L21_V001, 2` .

Task 2: VQA (`src/tasks/vqa.py`)

Input: A question about a specific moment in a video. **Output format**: `video_id, frame_id, answer`

```
question text
  → encode_text(question)
  → search_clip(top_k=20)    (fewer — VLM inference is slow)
  → filter_by_objects(mode="boost")
  → for each of top-5 candidates:
      load keyframe JPEG
      → vlm_answer(image, question)
  → pick candidate with highest confidence
  → return (video_id, frame_id, answer)
```

VLM selection given GPU constraints:

Model	Quantization	VRAM	Local 8GB	T4 15GB
Qwen2-VL-7B	4-bit (bitsandbytes)	~5 GB	✅	✅
InternVL2-8B	4-bit	~5 GB	✅	✅
Qwen2-VL-7B	8-bit	~9 GB	❌	✅

Recommended: Qwen2-VL-7B with 4-bit quantization locally.

Prompt:

```
You are analyzing a single frame from a video.
Question: {question}
Answer concisely. If the question is in Vietnamese, answer in Vietnamese.
Answer:
```

> **Fallback** (no GPU): return top CLIP result with `answer = "[manual review]"`. The UI highlights these for the team to answer manually during competition.

Task 3: TRAKE (`src/tasks/trake.py`)

Input: Overall video description + ordered list of N event descriptions. **Output format**: `video_id, frame_id_1, frame_id_2, ..., frame_id_N`

Phase A — Video Retrieval (find the correct video)

Aggregate keyframe scores at the video level:

```
all_results = search_clip(query_vector, top_k=len(metadata))
# Group by video_id: take max CLIP score per video
video_scores = metadata.with_columns(...).group_by("video_id").agg(pl.col("score").max())
target_video = video_scores.sort("score", descending=True)["video_id"][0]
```

Phase B — Event Localization (find the exact keyframe for each event)

For each event in the ordered sequence, search within the target video only:

```
candidate_sets = []
for event_desc in events:
    vec = encode_text(event_desc)
    candidates = search_clip_in_video(vec, video_id=target_video, top_k=5)
    candidate_sets.append(candidates)
```

Temporal ordering constraint: enforce `frame_id_1 < frame_id_2 < ... < frame_id_N` . If greedy top-1 violates ordering, apply DP alignment to find the best monotonically increasing assignment from the candidate sets:

```
def dp_align(candidate_sets: list[list[dict]]) -> list[dict]:
    """
    Returns the highest-scoring assignment [c_1, ..., c_N] where
    c_i["frame_id"] < c_{i+1}["frame_id"] for all i.
    Maximize sum of individual CLIP scores.
    O(N * K^2) where K = candidates per event (small).
    """
```

> **Known limitation**: CLIP ViT-B/32 may lack precision for very short event windows (<10 frames). Future enhancement: add VLM verification for Phase B top candidates.

Phase 4: Streamlit UI

File: `app.py`

Sidebar Controls

- **Task selector**: `[KIS-T | VQA | TRAKE]`
- **Top-K slider**: 10–100 (default 50)
- **Object filter**: text input, comma-separated labels (e.g., `"Person, Food"`)
- **Temporal search**: toggle on/off

Main Panel — KIS-T & VQA

```
[ Query text input (large)                                          ]
[ Search ]

Results — 3-column grid:
┌─────────────────┐  ┌─────────────────┐  ┌─────────────────┐
│   [image]       │  │   [image]       │  │   [image]       │
│ L21_V001        │  │ L21_V003        │  │ L21_V002        │
│ frame: 90       │  │ frame: 411      │  │ frame: 858      │
│ score: 0.82     │  │ score: 0.78     │  │ score: 0.75     │
│ Person, Food    │  │ Person          │  │ Building        │
│ [✓ Submit]      │  │ [✓ Submit]      │  │ [✓ Submit]      │
└─────────────────┘  └─────────────────┘  └─────────────────┘

Submission string: [ L21_V001, 90 ]  [Copy]
```

For VQA: show VLM-generated answer below each thumbnail; allow manual override.

Main Panel — TRAKE

```
[ Overall video description                                         ]
Event 1: [ text input ]
Event 2: [ text input ]
[+ Add event]  [ Search ]

Phase A — Top videos:  [L21_V001 | 0.82]  [L21_V003 | 0.71]  ...
                       [ Select this video ]

Phase B — Event keyframes within L21_V001:
  Event 1: [img] frame 90    Event 2: [img] frame 261   ...
  [ Confirm & Copy ]

Submission: [ L21_V001, 90, 261, 411, 531 ]  [Copy]
```

Run:

```
conda run -n aic26 streamlit run app.py
```

Phase 5: Configuration (`src/config.py`)

Full replacement for `src/config.py` :

```
# src/config.py
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent.parent
DATA_DIR = BASE_DIR / "data"
INDEX_DIR = BASE_DIR / "indexes"

# ── Data paths (confirmed from actual directory structure) ──────────────
VIDEOS_DIR        = DATA_DIR / "raw_videos" / "video_L21"   # only L21 downloaded locally
KEYFRAMES_DIR     = DATA_DIR / "keyframes"                   # LXX_Vxxx/NNN.jpg
CLIP_FEATURES_DIR = DATA_DIR / "clip-features-32"            # LXX_Vxxx.npy, one per video, [N
MAP_KEYFRAMES_DIR = DATA_DIR / "map-keyframes"               # LXX_Vxxx.csv (n, pts_time, fps,
OBJECTS_DIR       = DATA_DIR / "objects"                     # LXX_Vxxx/NNN.json
MEDIA_INFO_DIR    = DATA_DIR / "media-info"                  # LXX_Vxxx.json (TBC)

# ── Index paths ─────────────────────────────────────────────────────────
FAISS_INDEX_PATH  = INDEX_DIR / "faiss_clip.index"
METADATA_PATH     = INDEX_DIR / "metadata.parquet"
OBJECTS_PATH      = INDEX_DIR / "objects.parquet"
MEDIA_INFO_PATH   = INDEX_DIR / "media_info.json"

INDEX_DIR.mkdir(parents=True, exist_ok=True)

# ── CLIP model (MUST match the model used to create the .npy features) ──
CLIP_MODEL_NAME = "ViT-B-32"    # confirmed: folder is "clip-features-32"
CLIP_PRETRAINED = "openai"      # required to match competition-provided embeddings
CLIP_DIM        = 512

# ── Retrieval ───────────────────────────────────────────────────────────
DEFAULT_TOP_K        = 100
OBJECT_CONF_THRESH   = 0.3      # min confidence for object detections to keep
CLIP_WEIGHT          = 0.7      # alpha: CLIP cosine score weight in fusion
OBJECT_BOOST_WEIGHT  = 0.3     # beta: object boost weight in fusion

# ── Temporal search ─────────────────────────────────────────────────────
TEMPORAL_WINDOW   = 20          # +/- N keyframes for temporal ordering check
TEMPORAL_KEYWORDS = [
    # English
    "before", "after", "then", "while", "during", "next", "followed by",
    # Vietnamese (ASCII fallback + unicode)
    "truoc", "sau", "tiep theo", "trong khi", "sau do", "roi",
    "trước", "tiếp theo", "sau đó", "rồi",
]

# ── Shot segmentation (placeholder — used for Batch 2 re-ingestion, not Batch 1) ──
SHOT_MIN_LENGTH        = 10
OPTICAL_FLOW_THRESHOLD = 5.0
EMBEDDING_BATCH_SIZE   = 32
```

Phase 6: Verification Checklist

After Phase 1 (Ingestion)

- [ ] `indexes/metadata.parquet` exists; row count = sum of all keyframes across all CSVs
- [ ] `faiss_idx` column in metadata is 0, 1, 2, ... N-1 (no gaps)
- [ ] `indexes/faiss_clip.index` loads; `index.ntotal` == metadata row count
- [ ] `indexes/objects.parquet` contains entries for available videos
- [ ] Smoke test: for `L21_V001/001.jpg` → look up `npy_row_idx=0`, load `.npy[0]` → check shape is `(512,)`

After Phase 2 (Query Pipeline)

- [ ] `encode_text("a person cooking")` returns shape `(512,)` float32 vector
- [ ] `search_clip(...)` returns dicts with `frame_id` matching values from CSV `frame_idx` column
- [ ] Object filter changes ranking when `"Person"` is specified
- [ ] Temporal search triggers on `"person picks fruit then puts it in a bag"`
- [ ] Temporal search does NOT trigger on `"person standing at a market stall"`
- [ ] Single query latency < 2 seconds on CPU

After Phase 3 (Task Heads)

- [ ] KIS-T: `frame_id` values match actual `frame_idx` from CSVs (not keyframe filename integers)
- [ ] VQA: returns non-empty `answer` string (with GPU) or `"[manual review]"` (without GPU)
- [ ] TRAKE: N `frame_id` values are in **strictly increasing order**

After Phase 4 (UI)

- [ ] Streamlit launches at `localhost:8501` without errors
- [ ] Keyframe thumbnails render correctly
- [ ] Submission string is correctly formatted for KIS-T, VQA, and TRAKE
- [ ] Copy to clipboard works

Resolved Open Questions

#	Question	Answer
1	CLIP `.npy` structure	One `.npy` per video in `clip-features-32/` — shape `[N, 512]`
2	Map-keyframes format	CSV per video: `n, pts_time, fps, frame_idx` — `frame_idx` is what to submit
3	Objects JSON keys	`detection_class_entities`, `detection_scores`, `detection_boxes` — all values are strings
4	GPU available	RTX 4060 8GB (local) + Colab T4 (limited) — use 4-bit quantized VLMs
5	Batch 2 timeline	No info yet — **All core features due 21/08**, fixes/tuning 28/08 and 04/09
6	Dataset scope	L21–L30, 873 videos; only L21 downloaded locally for now

---

Build Order Summary

#	Module	Unblocked when	Deadline	Est. effort
1	`src/config.py` update	Now	**21 Aug**	15 min
2	`src/offline_pipeline/metadata_loader.py`	CSVs present ✅	**21 Aug**	1–2 hrs
3	`src/offline_pipeline/build_index.py`	`.npy` files present ✅	**21 Aug**	1 hr
4	`src/offline_pipeline/objects_loader.py`	Objects present ✅	**21 Aug**	1–2 hrs
5	`main_ingest.py`	Steps 2–4	**21 Aug**	30 min
6	`src/online_pipeline/query_encoder.py`	CLIP installed, step 5	**21 Aug**	30 min
7	`src/online_pipeline/retrieval.py`	Step 6	**21 Aug**	1 hr
8	`src/online_pipeline/object_filter.py`	Steps 4, 7	**21 Aug**	1 hr
9	`src/online_pipeline/fusion.py`	Steps 7–8	**21 Aug**	30 min
10	`src/tasks/kis_t.py`	Step 9	**21 Aug**	1 hr

#	Module	Unblocked when	Deadline	Est. effort
11	`app.py` — basic UI (KIS-T)	Step 10	**21 Aug**	2–3 hrs
12	`src/online_pipeline/temporal_search.py`	Step 9	**21 Aug**	2–3 hrs
13	`src/tasks/vqa.py`	Step 10 + GPU	**21 Aug**	1–2 hrs
14	`src/tasks/trake.py`	Step 12	**21 Aug**	2 hrs
15	UI — VQA + TRAKE panels	Steps 13–14	**21 Aug**	2–3 hrs
16	Tuning, stress test, polish	All above	**04 Sep**	ongoing
Total				**~18–24 hrs**

> **Start now (steps 1–9)**: CLIP features + map-keyframes CSVs are already downloaded. You can build and verify the full ingestion + retrieval pipeline **today** without waiting for more downloads.